Common Core: Phase 1 RTL Design

Lab R4: Data Cache

Cache Micro-Architecture, Controller FSM and Performance Counters

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 18, 2026

Lab objective: Design a parameterized direct-mapped, write-back, write-allocate data cache with a controller FSM and performance counters, connect it to the Lab R3 memory through the same `mem_if` interface, and check it with a directed smoke testbench whose hit rates you predict by hand before you run them.

Duration	2 weeks (approximately 30–40 hours)
Prerequisites	Labs L0 and R1–R3
Designs	`dcache`, `dcache_pkg` (new); `mem_if`, `arb_mem` (reused from Lab R3, unchanged)
Reused in	V3 (verification), P3 (area, timing, SRAM discussion), T-RTL2 (multi-clock integration), T-MOD, T-DV1, T-DV2, T-AM1 and T-AM2 (model, testbenches and performance correlation)

Contents

1 Overview **3**

2 Cache Architecture **3**

2.1 Parameters and Address Fields . . . 3

2.2 Storage . . . 4

2.3 Policies . . . 4

2.4 Performance Metrics . . . 4

3 Design Specification **5**

3.1 System Structure . . . 5

3.2 Ports . . . 5

3.3 Behavior . . . 6

3.4 Controller . . . 6

3.5 Internal Block Diagram . . . 7

3.6 Out of Scope . . . 7

4 RTL Requirements **7**

5 Smoke Testbench **8**
5.1 Building Blocks . . . 8
5.2 Directed Tests . . . 9

6 Performance: Predict, Then Measure **10**
6.1 Miss Penalty From the RTL . . . 10
6.2 Hand Prediction . . . 10
6.3 Questions . . . 11

7 Stretch Goal: 2-Way Set-Associative Cache **11**

8 Requirements / Tasks **11**

9 Repository and Flow **12**
9.1 Files Created in This Lab . . . 12
9.2 File Lists and Reuse . . . 12
9.3 Makefile . . . 13
9.4 Commands . . . 13

10 Deliverables **13**

11 Assessment **14**

12 Review Questions **14**

13 References **15**

1 Overview

Memory is slow compared to the processor. A cache keeps recently used data close to the core, so most accesses complete in one cycle and only a few have to wait for main memory. In this lab you build the data cache that would sit between a RISC-V core's load/store unit and memory.

The cache is *blocking*: while it handles a miss, it does not accept new requests. This keeps the control logic manageable while still covering the central ideas of cache design: address decomposition, tag comparison, refill, write-back of dirty lines, flushing, and measuring performance with counters in the hardware.

The cache is the largest design of the common core and the one most later labs build on, so its micro-architecture is documented as carefully as it is coded.

Where this lab sits. It reuses `mem_if` and `arb_mem` from Lab R3 *in place*, through file lists, and the coding rules, lint flow and `<=` drive standard of Labs R1 and R2. The cache is verified with classes, constrained random stimulus and functional coverage in Lab V3, synthesized and analyzed for area and critical path in Lab P3, moved behind a clock-domain crossing in Lab T-RTL2, modeled at transaction level in Lab T-MOD, and its performance counters are correlated against that model in Labs T-AM1 and T-AM2. The hand predictions you make here are compared with measured numbers in Lab V3.

By the end of this lab you should be able to:

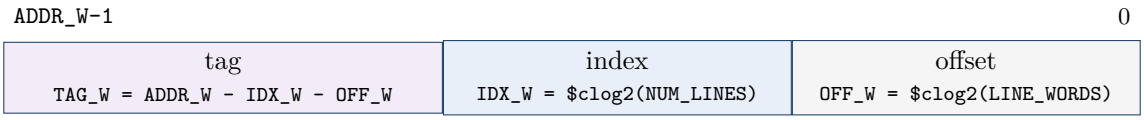
1. Split an address into tag, index and offset for any cache configuration, and explain how each parameter changes them.
2. Explain the placement, write, allocation, replacement and miss-handling policies of a cache and the trade-off behind each.
3. Implement a blocking write-back, write-allocate cache with refill, eviction and flush, controlled by a documented FSM.
4. Keep an existing valid/ready protocol unchanged on a block whose latency varies from one to many cycles.
5. Put performance counters in RTL and use them to measure hit rate, miss penalty and average memory access time.
6. Predict the hit rate of a directed access pattern by hand, and explain any difference from the measurement.

2 Cache Architecture

2.1 Parameters and Address Fields

Addresses are *word* addresses, as in Lab R3. A cache *line* holds `LINE_WORDS` consecutive words; the cache holds `NUM_LINES` lines.

Parameter	Default	Meaning
`ADDR_W`	10	Word address width (main memory: `2**ADDR_W` words)
`DW`	32	Data width
`LINE_WORDS`	4	Words per line (power of two, ≥ 2)
`NUM_LINES`	16	Number of lines (power of two, ≥ 2)



- The **offset** selects the word inside a line.
- The **index** selects the one line in the cache where this address may be stored (direct-mapped: exactly one place).
- The **tag** identifies which of the many memory lines that share this index is currently stored there.

With the defaults: `OFF_W = 2`, `IDX_W = 4`, `TAG_W = 4`. The cache stores $16 \times 4 = 64$ words of a 1024-word memory. Two addresses *conflict* when they have the same index and different tags: with the defaults, addresses that differ by a multiple of 64 words.

2.2 Storage

Per line the cache stores a valid bit, a dirty bit, a tag and `LINE_WORDS` data words:

```
logic                valid [NUM_LINES];
logic                dirty [NUM_LINES];
logic [TAG_W-1:0]    tag   [NUM_LINES];
logic [DW-1:0]       data  [NUM_LINES][LINE_WORDS];
```

Valid and dirty bits are control state and must be reset; tags and data are datapath arrays and are not reset (Lab R1, rule 6), so they live in their own `always_ff` block without a reset branch. All four are unpacked arrays of storage locations, chosen for the reasons discussed in Lab R3. How much area each of them costs, and why a real cache would use SRAM macros for tag and data, is measured in Lab P3.

2.3 Policies

Decision	This lab	Alternative
Placement	Direct-mapped: one possible line per address	Set-associative (stretch goal) or fully associative
Write hit	Write-back: update the cache only, set dirty	Write-through: update cache and memory
Write miss	Write-allocate: refill the line, then write	No-write-allocate: write memory only
Replacement	Not needed (one candidate)	LRU, random, pseudo-LRU
Miss handling	Blocking: one miss at a time	Non-blocking with miss status holding registers (MSHRs)

For each choice, `doc/design.md` must explain one advantage and one disadvantage compared to the alternative.

2.4 Performance Metrics

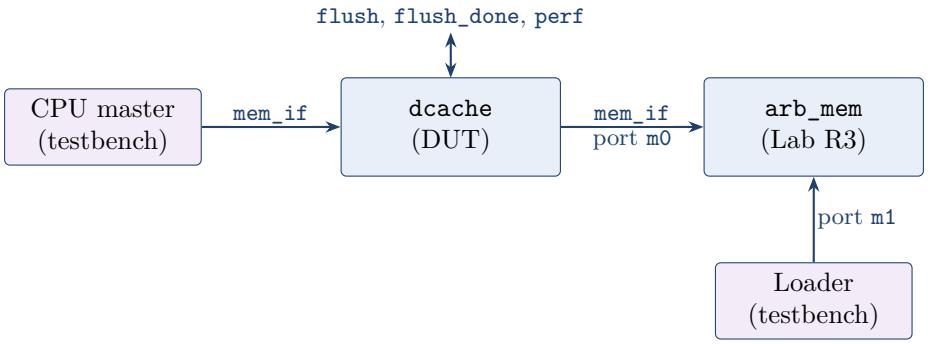
- **Hit rate** $= 1 - \mathit{misses}/\mathit{accesses}$.
- **Miss penalty**: extra cycles a miss costs. A clean miss costs one refill; a dirty miss costs an eviction plus a refill. Both depend on `LINE_WORDS` and on your controller.
- **Average memory access time**:

$$AMAT = hit\ time + miss\ rate \times miss\ penalty$$

The formula is a model. Your RTL counters measure the real thing, and the difference between them (cycles the formula does not count) is one of the questions of this lab.

3 Design Specification

3.1 System Structure



- The CPU side of the cache is a `mem_if` slave; the memory side is a `mem_if` master connected to port `m0` of the Lab R3 `arb_mem`, instantiated with `AW = ADDR_W`.
- Port `m1` of `arb_mem` belongs to a testbench *loader*. It preloads the memory before the test. It must stay idle while the cache is in use: a second master changing memory behind the cache's back would make the cached copy stale (that problem, coherence, is out of scope).
- `mem_if` and `arb_mem` are **not copied**. They stay in `hw/ip/arb_mem/rtl/` and are pulled in through file lists (Section 9). A bug fixed there (for example in Lab V2) is fixed for the cache too.

3.2 Ports

```
module dcache #(
  parameter int ADDR_W     = 10,
  parameter int DW         = 32,
  parameter int LINE_WORDS = 4,
  parameter int NUM_LINES  = 16
) (
  input  logic              clk, rst_n,
  mem_if.slave              cpu,        // requests from the core
  mem_if.master             mem,        // requests to main memory
  input  logic              flush,      // one-cycle pulse, CPU side idle
  output logic              flush_done, // one-cycle pulse when finished
  output dcache_pkg::perf_t perf        // performance counters
);
```

Both interfaces have `AW = ADDR_W` and the same `DW`. `perf_t` is a packed struct of four 32-bit counters, declared in `dcache_pkg`:

```
package dcache_pkg;
  typedef struct packed {
    logic [31:0] accesses;      // accepted CPU requests
    logic [31:0] misses;        // refills caused by CPU requests
    logic [31:0] writebacks;    // dirty lines written to memory (incl. flush)
    logic [31:0] stall_cycles;  // cycles with cpu.req_valid && !cpu.req_ready
```

```
  } perf_t;
endpackage
```

A miss request is counted once in `accesses` (when it is finally accepted) and once in `misses` (when its refill starts), so *hit rate* $= 1 -$ `misses/accesses` holds directly. The counters are reset by `rst_n` and wrap at 2^{32}; document both choices.

3.3 Behavior

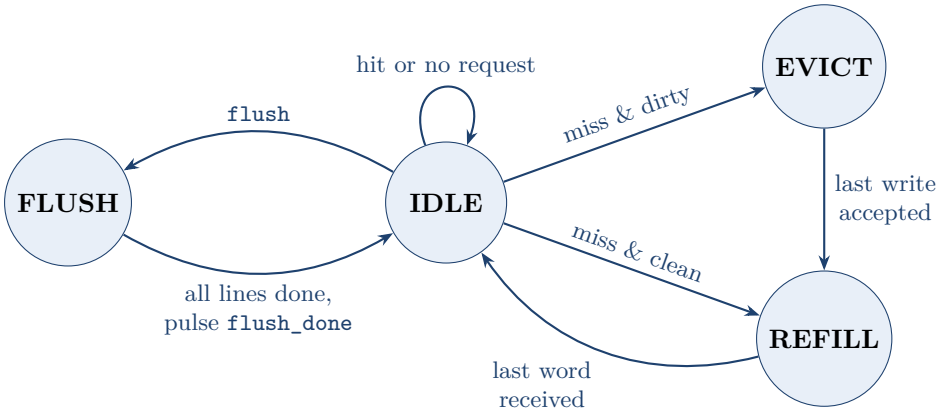
The CPU-side protocol is exactly the `mem_if` protocol from Lab R3. The cache keeps that protocol by **accepting a request only when it hits**:

1. In the idle state, `cpu.req_ready` is high when the requested address hits. A read hit returns its data one cycle after acceptance; a write hit updates the line and sets its dirty bit.
2. On a miss, `cpu.req_ready` stays low. If the line at that index is valid and dirty, the cache first writes its `LINE_WORDS` words back to memory (eviction). It then reads the `LINE_WORDS` words of the new line (refill), sets valid, clears dirty and stores the new tag.
3. The cache returns to the idle state. The request, still held by the master (protocol rule 2), now hits and is accepted.
4. On `flush`, the cache writes back every dirty line, clears all valid and dirty bits, and pulses `flush_done`.

On the memory side the cache is a master and must obey the same rules it relies on: its `mem.req_valid` must not depend combinationally on `mem.req_ready` (protocol rule 3), and it holds a request stable until it is accepted. `cpu.req_ready` depends combinationally on `cpu.req_valid` and the address (through the tag compare), which rule 3 allows.

3.4 Controller

The suggested controller has four states. You may add states (for example, to wait for the last refill response), but justify every state in `doc/design.md`.



Design notes:

- **Two counters in REFILL.** Requests and responses are decoupled by one cycle (Lab R3, protocol rule 5). Count issued read requests and received responses separately; leave REFILL when the last *response* has been written into the line.
- **Addresses.** Refill reads `{req_tag, idx, word_cnt}`; eviction writes `{tag[idx], idx, word_cnt}` — the *stored* tag, not the requested one.

- **FLUSH.** Walk the indices; for a dirty line, write its words back (you may reuse the EVICT datapath); clear valid and dirty. A flush with no dirty line still takes `NUM_LINES` cycles or fewer; state what yours takes.
- **FSM style.** Lab R1, rule 11: state register in `always_ff`, next state in `always_comb` with `state_next = state` first, a `default` item, an enum with an explicit width (it may live in `dcache_pkg`).

3.5 Internal Block Diagram

Figure 1 shows the boxes you design inside `dcache` and how they connect. The address of the CPU request is split into tag, index and offset. The index selects one entry of the tag/valid/dirty array; the stored tag is compared with the requested tag (`hit`). The index and offset select a word of the data array, which feeds the read data register (a read hit) and the memory write data (an eviction). The controller FSM decides from `hit` and the dirty bit what happens; the word counters step through the line during EVICT and REFILL; the miss registers keep the index and the requested tag of the miss; the memory request mux builds the memory address from the *stored* tag (EVICT) or the *requested* tag (REFILL). The performance counters only observe events. Draw the same diagram, with your own signal names, in `doc/design.md`.

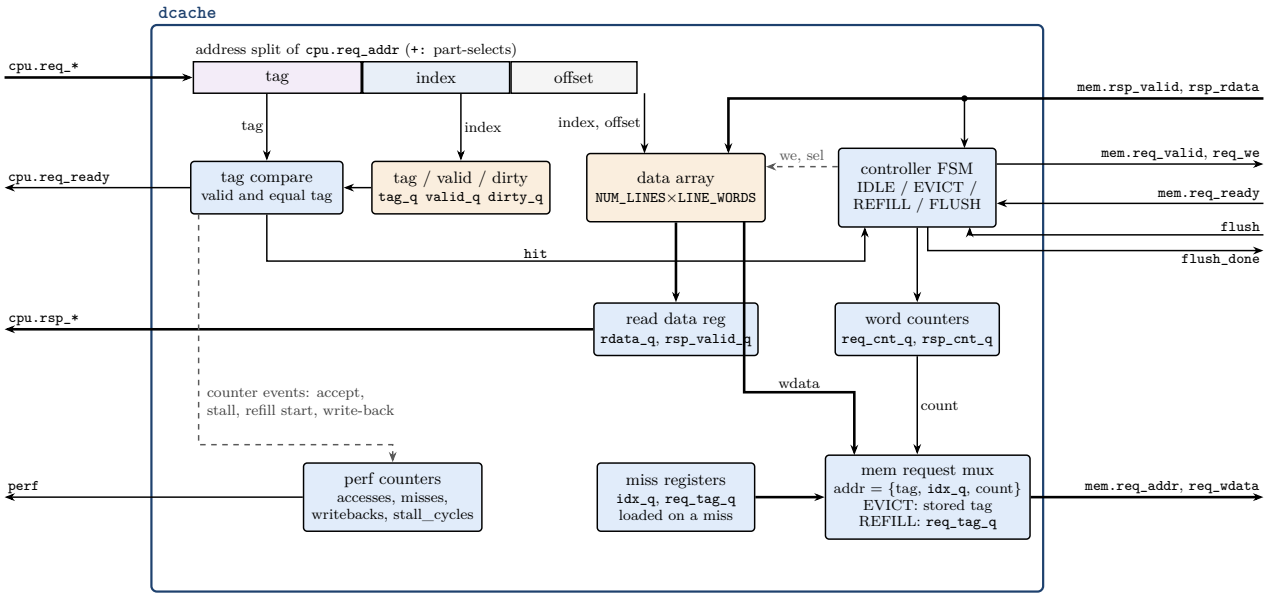


Figure 1: Inside `dcache`: the boxes to design. Blue: logic and registers; orange: storage arrays. Solid arrows are data and handshake signals (thick: buses), dashed arrows control (write enables, counter events). The CPU-side `mem_if` slave is on the left, the memory-side `mem_if` master on the right.

3.6 Out of Scope

Byte enables, non-blocking misses, a second master writing memory during operation, and coherence. Note these as limitations in the README.

4 RTL Requirements

- All Lab R1 coding rules apply. The design must be lint-clean; every waiver in `lint/dcache.vlt` has a justification.

- Field widths are `localparam`s derived with `$clog2`; no number in the RTL may depend on the default parameter values. Use `+:` part-selects to extract fields: `addr[OFF_W +: IDX_W]`.
- The hit signal is computed combinationally from the stored tag and valid bit of the indexed line.
- Read data is registered: `cpu.rsp_rdata` comes from a flip-flop, not directly from the data array.
- Refill responses may arrive in consecutive cycles. The design must not assume any gap between them.
- Performance counters are part of the RTL, not the testbench. They are the only performance instrumentation the later labs use (V3, T-AM1, T-AM2), so their definitions in `dcache_pkg` are a contract.
- The design must work, unchanged, for at least the configurations `LINE_WORDS` $\in \{2, 4, 8\}$ and `NUM_LINES` $\in \{8, 16, 32\}$.

Not in this lab. Design assertions for the cache (state/traffic consistency, bounded latency) are written in Lab V3; synthesis, the critical path through the tag compare and the SRAM discussion are Lab P3. Your miss-penalty derivation below is what Lab V3 uses to set the latency bound.

5 Smoke Testbench

Test term — Smoke test. A short directed test of the most basic behavior — reset, a hit, a clean miss, a dirty miss — compared with expected results. It answers "is the cache alive and does the flow work?" in seconds, for every parameter set. A pass proves little about correctness; a failure means nothing else is worth running. `smoke` stays the first test of the Lab V3 regression.

Test term — Directed test. A hand-written stimulus sequence with a known expected result, aimed at one feature ("write a line, evict it, read it back from memory"). Easy to debug, because you know what each access must do, but it only checks what its author thought of. Every `+TEST=` of this lab is directed; constrained-random stimulus follows in Lab V3.

The smoke testbench `tb/tb_dcache.sv` instantiates the whole system of the block diagram: three `mem_if` instances (`cpu_if`, `dmem_if`, `ld_if`), `dcache` and `arb_mem`. It is directed and self-checking and uses the techniques of Labs R2 and R3: one `initial` block, inputs driven with `<=`, interface signals accessed by name, static arrays as the model, a watchdog, the configuration printed at the start and one `TEST PASSED` / `TEST FAILED` line at the end.

5.1 Building Blocks

Part	Responsibility
`ld_write(addr, data)`	Write one word through the loader port `m1`. Used only before the cache is active, to preload a known pattern `f(a) = a ^ 32'h5A5A_0000` into every word
Model	Static array `model[2**ADDR_W]` initialized with the same pattern and updated by every accepted CPU write
`cpu_read(addr)`	Drive a read, hold it until accepted, count the wait cycles, check `rsp_valid` one cycle later and the data against the model
`cpu_write(addr, data)`	Drive a write, hold it until accepted, update the model
`do_flush()`	Pulse `flush` with the CPU side idle, wait for `flush_done` (with a timeout)
`snap()` / `expect_delta()`	Copy `perf` before an operation; afterwards check the change of each counter against the expected outcome

Checking counter *deltas* is how this testbench knows whether an access was a hit, a clean miss or a dirty miss: the designer knows the expected outcome of each directed access and checks that the hardware's own counters agree. (Lab V3 builds an independent prediction of every access and classifies the memory traffic; that is verification, not part of the smoke test.)

```
dcache_pkg::perf_t p0;
task automatic snap(); p0 = perf; endtask

task automatic expect_delta(string what, int acc, int miss, int wb);
  if (perf.accesses   - p0.accesses   != acc  ||
      perf.misses     - p0.misses     != miss ||
      perf.writebacks - p0.writebacks != wb) begin
    $display("ERROR %s: d_acc=%0d d_miss=%0d d_wb=%0d, expected %0d/%0d/%0d",
             what, perf.accesses - p0.accesses, perf.misses - p0.misses,
             perf.writebacks - p0.writebacks, acc, miss, wb);
    errors++;
  end
endtask

// directed hit / clean miss / dirty miss, default configuration
localparam logic [ADDR_W-1:0] A = 'h010;             // index 4, tag 0
localparam logic [ADDR_W-1:0] B = A + LINE_WORDS * NUM_LINES; // same index
snap(); cpu_read(A);          expect_delta("cold read",  1, 1, 0);
snap(); cpu_read(A + 1);      expect_delta("same line",  1, 0, 0);
snap(); cpu_write(A, 'hCAFE); expect_delta("write hit",  1, 0, 0);
snap(); cpu_read(B);          expect_delta("dirty miss", 1, 1, 1);
snap(); cpu_read(A);          expect_delta("refetch",    1, 1, 0); // data = CAFE
```

The last read proves that the write-back really reached memory: the line was evicted, so the value `CAFE` can only come from `arb_mem`.

5.2 Directed Tests

Test term — Corner-case test. A directed test at the edges of the specification, where most bugs live: a flush with no, some and all lines dirty, a refill whose requested word is not word 0, a write miss that first evicts a dirty line. `flush` and `refill_offset` are corner-case tests; `sequential` and `conflict` are directed performance patterns.

+TEST=	Scenario and expected result
`smoke`	The hit / clean miss / dirty miss sequence above, for reads and for writes (a write miss refills, then the write hits and sets dirty). Wait cycles: 0 for a hit, the clean or dirty miss penalty otherwise
`flush`	Flush with no dirty line (`writebacks` unchanged), with two dirty lines (+2) and with all lines dirty (+`NUM_LINES`); after each flush, a read of a previously written address is a clean miss and returns the written value
`sequential`	After a flush: reads with stride 1 over four times the cache capacity, starting at a line boundary. Compare the counters with your prediction
`conflict`	After a flush: alternating reads of two addresses with the same index and different tags, $2n$ accesses; then the same with every access a write. Compare with your prediction
`refill_offset`	Clean misses on each word offset of a line (the requested word is not always word 0); the data must be correct

What each test is meant to show:

- `smoke` — the three outcomes of an access exist and are counted correctly, and a write-back really reaches memory.
- `flush` — the flush writes back exactly the dirty lines, invalidates everything, and its duration follows your formula.
- `sequential` — spatial locality: one miss per line; the measured hit rate is the one you predicted.
- `conflict` — the weakness of a direct-mapped cache: two lines on one index evict each other on every access.
- `refill_offset` — the refill fills every word of the line at the right position, whichever word was requested.
- `all` — all of the above in one run (the default).

The default `TEST=all` runs all of them. Every test must pass for the default configuration and for `PARAMS="LINE_WORDS=2 NUM_LINES=32"` and `PARAMS="LINE_WORDS=8 NUM_LINES=8"`: write the expected counter values as expressions of the parameters, never as numbers.

6 Performance: Predict, Then Measure

A cache designer is expected to know what the hardware will do before the simulator says so. Before running `sequential` and `conflict`, *predict* the counters from the parameters and the access pattern and write the prediction in `doc/design.md`. Then run, read the performance counters (the testbench prints them at the end of each test) and explain every difference.

6.1 Miss Penalty From the RTL

Derive the clean-miss and dirty-miss penalties of *your* controller as functions of `LINE_WORDS`, by counting the cycles of each state and transition, for example $T_{\text{clean}}(L) = L + c_1$ and $T_{\text{dirty}}(L) = 2L + c_2$. Then measure them: the wait cycles of a single directed miss, and the change of `stall_cycles`, must equal your formula exactly for $L = 2, 4, 8$. A difference means either the formula or the RTL is not what you think it is.

6.2 Hand Prediction

Hints for the default configuration (do the general derivation yourself):

- **Sequential, stride 1, over $4\times$ capacity, after a flush:** every line is touched once, so one miss per line and `LINE_WORDS` $- 1$ hits; nothing is dirty, so no write-backs.
- **Conflict, alternating reads A, B, A, B, . . . :** every access evicts the line the next access needs.
- **Conflict, alternating writes:** as above, but now every evicted line is dirty.
- **Stall cycles** follow from the number of misses and the penalties you derived.

Test	Accesses	Predicted hit rate	Measured hit rate	Write-backs pred. / meas.	Stall cycles pred. / meas.	AMAT, formula / measured
`sequential`						
`conflict` (reads)						
`conflict` (writes)						

6.3 Questions

1. With the same total capacity, predict and then measure the `sequential` hit rate for `LINE_WORDS=2, NUM_LINES=32` and `LINE_WORDS=8, NUM_LINES=8`. Why do longer lines help this pattern? What kind of pattern would they hurt? (Random patterns are measured in Lab V3; a systematic design-space exploration is Lab T-AM1.)
2. How do the clean and dirty miss penalties depend on `LINE_WORDS`? Show both formulas and both measurements.
3. Does the measured average latency per access match the AMAT formula? If not, which cycles does the formula not count (think of the response cycle, the FSM transitions, the first refill request)?
4. Change the conflict pattern to A, A+1, B, B+1, ... and predict the hit rate before you run it.

7 Stretch Goal: 2-Way Set-Associative Cache

Add a parameter `WAYS` (1 or 2). With `WAYS=2`, each index selects a *set* of two lines; the tag is compared in both ways in parallel, and on a miss the least recently used way is replaced (one LRU bit per set).

- Use `generate` to create the per-way storage and tag comparators; keep `WAYS=1` identical in behavior to the direct-mapped cache.
- At most one way may hit. Make it impossible by construction and explain why; the assertion for it is added in Lab V3.
- Rerun `conflict` with two alternating tags, predict and explain the change in hit rate. Then predict, and measure, three alternating tags on one set with LRU.
- Verification of the stretch goal is optional in Lab V3; its area and critical path compared with the direct-mapped cache of the same capacity are measured in Lab P3.

8 Requirements / Tasks

1. Create the branch `lab/r4-dcache` and `hw/ip/dcache/` with the layout of Section 9. Do not copy anything from `hw/ip/arb_mem/`.
2. Write `doc/spec.md`: parameters, ports, `perf_t` counter definitions, behavior and out-of-scope items.
3. Compute the address fields for the default configuration and for two others of your choice; put the table in `doc/design.md`.
4. Write `rtl/dcache_pkg.sv` and `rtl/dcache.sv` according to the specification and the RTL requirements.
5. Write `rtl/dcache.f`, `tb/tb.f`, the lint wrapper and `lint/dcache.vlt`; `make lint` is clean.
6. Write `tb/tb_dcache.sv` with all tests of Section 5.2; `make sim` passes for the three configurations.
7. Derive the miss penalties, make the hand predictions, measure, and fill the performance table and the questions in `doc/design.md`.
8. Document the policies (one advantage and one disadvantage each) and the controller FSM with a justification of every state and transition.
9. Confirm that `make -C $REPO_ROOT check-all` passes, including `arb_mem` from Lab R3, update `README.md`, open the merge request and tag `r4-done` after review.

10. Optional: the 2-way stretch goal of Section 7.

Test term — Sanity test. `make check-all`: lint and the smoke test of every block in the repository, run together. It shows that adding `dcache` (and any fix in `hw/ip/arb_mem`) did not break a block that used to work. A pass proves nothing new about the cache; a failure blocks the merge request.

9 Repository and Flow

9.1 Files Created in This Lab

```
hw/ip/dcache/
|-- README.md                 how to run, results, known limitations
|-- Makefile
|-- doc/
|   |-- spec.md               ports, counters, behavior, out of scope
|   `-- design.md             fields, policies, FSM, penalties, predictions
|-- rtl/
|   |-- dcache_pkg.sv         perf_t (and the state enum, if you like)
|   |-- dcache.sv
|   `-- dcache.f              -f arb_mem/rtl/mem_if.f + pkg + dcache
|-- lint/
|   |-- dcache.vlt            waivers, each with a justification
|   |-- dcache_lint_top.sv    wrapper with plain ports (lint only)
|   `-- lint.f
`-- tb/
    |-- tb_dcache.sv          directed smoke testbench
    `-- tb.f                  -f dcache.f + -f arb_mem/rtl/arb_mem.f + tb
```

Nothing in `hw/ip/arb_mem/` changes in this lab. If you find a bug there, fix it in `hw/ip/arb_mem/rtl/` with its own commit and rerun the Lab R3 smoke test.

9.2 File Lists and Reuse

```
// hw/ip/dcache/rtl/dcache.f  -- synthesizable files of the cache only
-f ${REPO_ROOT}/hw/ip/arb_mem/rtl/mem_if.f
${REPO_ROOT}/hw/ip/dcache/rtl/dcache_pkg.sv
${REPO_ROOT}/hw/ip/dcache/rtl/dcache.sv

// hw/ip/dcache/tb/tb.f  -- the TB instantiates arb_mem as main memory
-f ${REPO_ROOT}/hw/ip/dcache/rtl/dcache.f
-f ${REPO_ROOT}/hw/ip/arb_mem/rtl/arb_mem.f
${REPO_ROOT}/hw/ip/dcache/tb/tb_dcache.sv
```

The cache itself needs only the interface, so `dcache.f` includes `mem_if.f` and not the memory; Lab P3 synthesizes `dcache` alone from this list. The testbench needs both, so `mem_if.sv` is reached twice — a *diamond dependency*. `flow/scripts/flist_expand.sh` must therefore drop repeated entries (first occurrence wins), and the flow must pass the expanded list to the tools. If your Lab L0 script does not do this yet, add it generically (one `awk '!seen[$0]++'` is enough) and rerun `check-all`. Dependency managers such as FuseSoC and Bender solve exactly this problem with a dependency graph instead of nested lists.

9.3 Makefile

```
# hw/ip/dcache/Makefile
IP          := dcache
DUT_TOP     := dcache
TB_TOP      ?= tb_dcache
RTL_FLIST   := rtl/dcache.f
TB_FLIST    ?= tb/tb.f
LINT_TOP    := dcache_lint_top      # interface ports, as in Lab R3
LINT_FLIST  := lint/lint.f
PARAMS      ?= ADDR_W=10 DW=32 LINE_WORDS=4 NUM_LINES=16
TEST        ?= all                  # every directed test (common.mk default: smoke)
include $(REPO_ROOT)/flow/mk/common.mk
include $(REPO_ROOT)/flow/mk/lint.mk
include $(REPO_ROOT)/flow/mk/sim_verilator.mk
```

9.4 Commands

```
source env.sh
cd hw/ip/dcache
make lint
make sim                                            # TEST=all, defaults
make sim TEST=conflict WAVES=1 && make waves
make sim PARAMS="ADDR_W=10 DW=32 LINE_WORDS=2 NUM_LINES=32"
make sim PARAMS="ADDR_W=10 DW=32 LINE_WORDS=8 NUM_LINES=8" TEST=sequential
make -C $REPO_ROOT check-all
```

On the Synopsys cloud: The commands are the same; `env.sh` selects `SIM=vcs` and `LINT_TOOL=spyglass`. SpyGlass (`make lint LINT_TOOL=spyglass`) and VC SpyGlass (`LINT_TOOL=vcst`) lint the same wrapper through `LINT_TOP`/`LINT_FLIST`; expect `W240` ("input `cpu.clk` declared but not read") for the clock and reset carried by the `mem_if` modports, a justified waiver in `lint/dcache.awl` (SpyGlass) and `lint/dcache.vcst.tcl`, as in Lab R3. `make sim WAVES=1` dumps FSDB and `make waves` opens Verdi: show `state_q` as an FSM and the two interface instances as signal groups to see an eviction followed by a refill. VCS is four-state, so a tag or data word that is used before it was ever written shows X. Verilator 5.013 on the cloud is older than the program minimum (5.020): it lints the cache clean, but it can let the `always_ff` blocks of the DUT see a testbench's `<=` drive in the same edge, so a request looks one cycle early and every miss one wait cycle short — simulate this lab with VCS.

Each `PARAMS` set builds into its own directory, for example:

```
build/dcache/ADDR_W-10_DW-32_LINE_WORDS-2_NUM_LINES-32/sim/
```

No new make targets are introduced. Synthesis of the cache is done in Lab P3.

10 Deliverables

The merge request from `lab/r4-dcache` contains:

- The RTL, file lists, lint wrapper and waivers, smoke testbench and Makefile; `make lint sim` passes for the three configurations, and `check-all` passes.
- `doc/spec.md`: ports, counter definitions, behavior, out of scope.

- `doc/design.md`:
  - the address field table for three configurations;
  - the policy discussion;
  - the controller state diagram with a justification of every state, and a block diagram of the datapath (tag compare, data array, refill and write-back address);
  - the miss-penalty formulas and their measurement;
  - the performance table with predictions, measurements, and the answers to the questions;
  - the stretch goal, if done.
- `README.md`: how to run, results per configuration, known limitations (out-of-scope items), and how the block depends on `hw/ip/arb_mem`.

11 Assessment

Criterion	Weight
Functional correctness across configurations (smoke tests pass)	25%
Micro-architecture: address fields, controller FSM, memory-side protocol	20%
RTL quality: coding rules, parameterization, lint clean, counters in RTL	10%
Smoke testbench: directed hit/miss/flush checks via counter deltas	15%
Performance: miss-penalty derivation, hand predictions, analysis	20%
Documentation and reuse: design notes, policies, file lists without copies	10%
Total	100%

The stretch goal adds up to 10 bonus points.

12 Review Questions

1. For `ADDR_W=12, LINE_WORDS=8, NUM_LINES=64`, what are the tag, index and offset widths? What is the capacity in words?
2. Why can the cache keep the Lab R3 `mem_if` protocol unchanged, even though a miss takes many cycles?
3. Why must valid and dirty bits be reset, but not the tag and data arrays?
4. Why does the eviction use the stored tag and the refill the requested tag? What happens if you swap them?
5. Your cache always misses. Does the data check of your smoke testbench find it? What does?
6. A write is lost because a dirty bit is never set. Which of your directed tests finds it, and why does a read through the cache right after the write not find it?
7. Why are refill requests and refill responses counted separately?
8. What would break if the loader wrote memory while the cache was active? What is this problem called?

9. Why does the testbench file list include `arb_mem.f`, while `dcache.f` includes only `mem_if.f`?
10. Which cycles does the AMAT formula leave out compared with your `stall_cycles` counter?

13 References

- Harris and Harris, *Digital Design and Computer Architecture: RISC-V Edition*: chapter on memory systems.
- Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*: memory hierarchy design, cache performance.
- Onur Mutlu, Digital Design and Computer Architecture lectures: caches and memory hierarchy.
- David Wentzlaff, Computer Architecture (Coursera): cache organization and performance modules.
- IEEE Std 1800-2017: Clause 7 (structures, arrays), Clause 26 (packages), Clause 27 (generate constructs).
- CVA6 and Ibex: open-source RISC-V cores with data caches and performance counters (for reading, not copying).